

Мишучков Егор Валерьевич

Программирование и Data Science. CV. ML.

Россия, Москва | +79162834890 | multogor@gmail.com | t.me/multogor | <https://github.com/ggrgrtr>

Курс 3, РТУ МИРЭА — Информатика и Вычислительная техника: Интеллектуальные системы обработки и передачи информации.

Прохожу основное обучение в Школе21 от Сбера по направлению Data Science.

Языки:

- Python (Основной)
- C++ - базовый уровень, ООП, работа с файлами
- Java - базовый уровень

Стек:

- PyTorch, TorchVision, OpenCV, NumPy, Matplotlib.pyplot, SQLite
- Apscheduler, Shutil, Tkinter, Pandas, Threading

Инструменты:

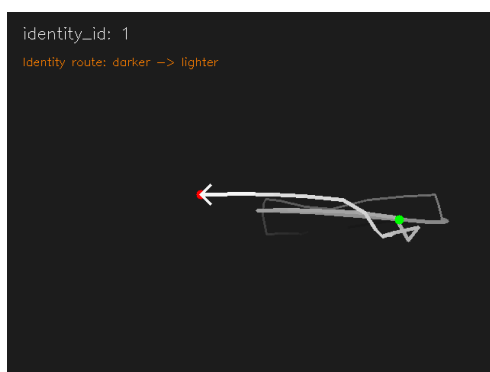
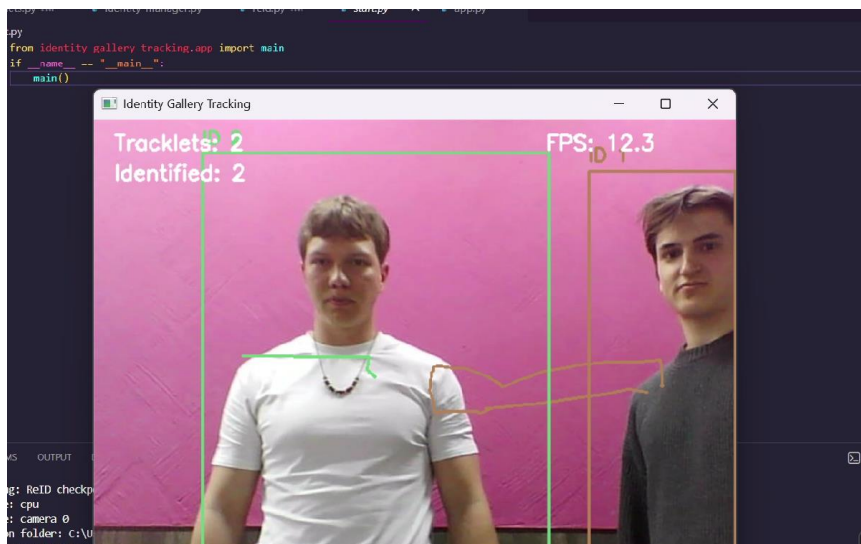
- Git & GitHub
- VS Code, PyCharm, Google Colab
- Sql СУБД

Проекты:

Pet-Project: People YOLO-detection, tracking and ReID-embedding

- Детектирование человека через архитектуру YOLO. Отслеживание перемещения с помощью системы обновляемых треков. Определение, сравнение и подтверждение трека и личности за счет вычисления вектора признаков на основе ReID-модуля ResNet50, хранение личностей, Калман-предсказание маршрута.
- Использовал PyTorch, OpenCV, Threading, Ultralytics, TorchVision.

Результат: Рабочая система отслеживания человека на видео или онлайн трансляции. Назначение ID и ведение tracklet-истории. ~3075 строк, 13 files, 12 classes



Репозиторий + видео:

- [ReadMe.md](#)

Курсовая работа. C++. Создание, изменение и загрузка базы данных.
Работа с динамической памятью и текстовыми файлами.

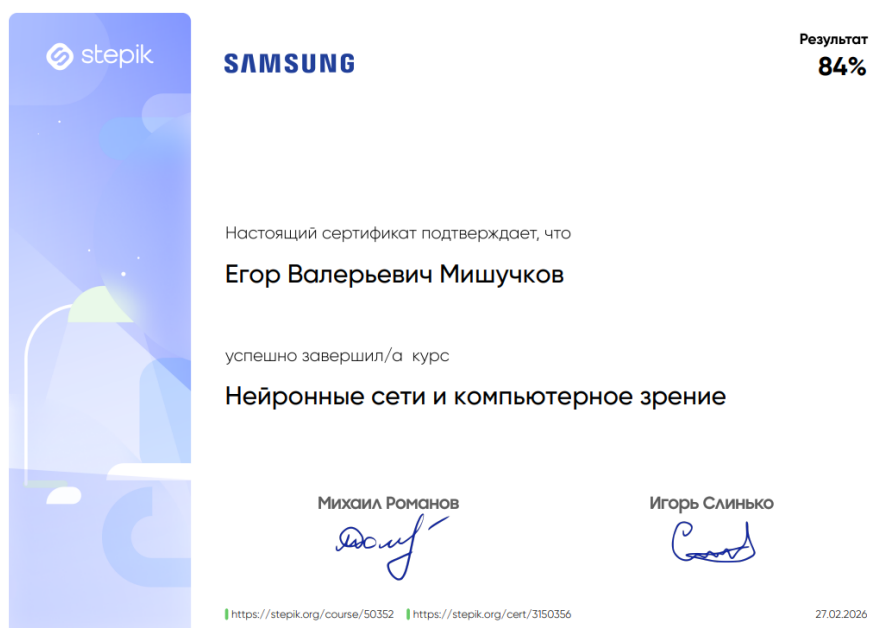
- [Описание_курсовая](#)

Java. Компьютерная 2-d игра с покадровой визуализацией спрайтов.
Танки:

- [ReadMe.md](#)

Дополнения:

- ❖ Финалист кейс-чемпионата "Траектория" по треку "Робототехника и мехатроника"
- ❖ Участник IT-акселератора Цифра по направлению IT-агрегатор курсов.
- ❖ Прошел онлайн-курс "Нейронные сети и компьютерное зрение"



Obsidian, древо знаний, Нейронные сети:

- [Obsidian_NN-CV](#)



Loss-Function

Пакетный градиентный спуск (Batch Gradient Descent): вычисляет градиент функции потерь по всему обучающему набору данных. Это точно, но может быть очень медленным на больших наборах данных.

Стохастический градиентный спуск (Stochastic Gradient Descent, SGD): вычисляет градиент для каждого обучающего примера по отдельности и обновляет параметры. Это быстро, но изменения параметров могут быть очень "шумными".

Мини-пакетный градиентный спуск (Mini-batch Gradient Descent): компромисс между двумя предыдущими методами, вычисляет градиенты на небольших группах обучающих примеров.

L2-регуляризация

Не обнуляет значения

$$P_n(x) = \sum^n w_i x^i - \text{Полином}$$

$$Loss = \sum^n [P_n(x_i) - y_i]^2 + \lambda \sum$$

- λ (лямбда) - регуляризационный параметр (гиперпараметр), который контролирует силу регуляризации. Он выбирается заранее или может быть настроен в процессе обучения модели.
- n - количество признаков в модели.
- w_i - вес (коэффициент) i -го признака W .
- y - target

Без Регуляризации:

$$P_n(x) = \sum_{i=0}^n a_n x^n$$

$$J = \sum (P_n(x_i) - y_i)^2$$

Activation function

Чувствительна к выбросам и несбалансированным данным

SOFTMAX

$$SM_i(y_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

$$z_i = w_i \cdot x_i + b_i$$

$$\sum SM_i = 1, SM_i \in [0, 1]$$

$$\frac{\partial SM_i}{\partial z_i} = \left[\frac{z_i}{\sum_j z_j} + \frac{z_i(1-z_i)}{\sum_j z_j} \right] = \frac{e^{z_i} - e^{z_i} \cdot SM_i}{(e^{z_i})^2} = \frac{e^{z_i} - e^{z_i} \cdot SM_i}{e^{z_i}} = 1 - SM_i$$